

Zizzet AI Lead Recovery Engine

AI Backend Developer Intern — Technical Screening Task

Objective

Build a simplified Zizzet backend feature that identifies inactive or high-intent leads, analyzes the lead and conversation history with an LLM, and recommends the next best action plus a personalized follow-up.

1. Core Task

Problem: Businesses lose revenue when interested leads stop responding or are not followed up at the right time. Build an API service that turns lead/conversation data into a structured recovery recommendation.

Example input

```
{ "tenant_id": "business_001", "lead_id": "lead_1024", "customer": {"name": "Arun Kumar", "phone": "+919876543210"}, "lead": {"source": "whatsapp", "status": "contacted", "created_at": "2026-09-20", "last_contacted_at": "2026-09-25"}, "conversation": [ {"role": "customer", "message": "I am interested in your CRM."}, {"role": "agent", "message": "How many users do you need?"}, {"role": "customer", "message": "Around 25 users. What is the pricing?" } ] }
```

Expected output

```
{ "lead_score": 86, "priority": "high", "intent": "purchase", "stage": "pricing_interest", "summary": "Customer is evaluating a CRM for a 25-member team.", "next_best_action": "Send pricing and schedule a demo", "follow_up_channel": "whatsapp", "follow_up_message": "Hi Arun! Just following up on your CRM requirement...", "do_not_contact": false }
```

2. Required APIs

Endpoint	Purpose
POST /api/v1/leads/analyze	Analyze lead + conversation
GET /api/v1/leads/{lead_id}/analysis	Retrieve latest analysis
POST /api/v1/leads/{lead_id}/follow-up	Generate follow-up
POST /api/v1/webhooks/leads	Receive lead event and trigger processing

3. Technical Requirements

- **Python + FastAPI** backend.
- **PostgreSQL** preferred; SQLite is acceptable for a lightweight submission.
- Use any major **LLM provider** such as OpenAI, Gemini, Anthropic, or Ollama.
- Return and validate **structured AI output** using Pydantic or an equivalent schema.
- Use a **background task/job queue** for webhook-triggered AI processing.
- Implement **tenant isolation**; a tenant must never access another tenant's lead or analysis.
- Implement **idempotency**; duplicate webhook events must not create duplicate processing.
- Handle invalid input, LLM failures, malformed AI responses, and database errors.
- If a customer says 'STOP' or otherwise opts out, set **do_not_contact=true** and do not generate a follow-up.
- A real WhatsApp API is not required; use a **mock messaging provider/interface**.

4. Suggested Architecture

FastAPI → validation → PostgreSQL / job queue → AI worker → LLM → structured-output validation → PostgreSQL → follow-up service.

5. Evaluation Cases

Case	Scenario	Expected
A	50 employees + asks for pricing	High priority / purchase intent
B	Only asks what the product does	Low priority
C	Requests a demo/call tomorrow	Demo/call as next action
D	Customer says: "STOP. Don't message me again."	do_not_contact=true; no follow-up
E	Same webhook event submitted twice	One job / one processing result

6. Submission Requirements

- GitHub repository with clean, readable code.
- Working API with Swagger/OpenAPI documentation.
- README: setup, architecture, environment variables, API usage, AI approach, assumptions and limitations.
- .env.example — never commit real API keys.
- Dependency file such as requirements.txt.
- Automated tests covering core APIs, validation/error cases and at least one AI/service case.
- Deployment is preferred; otherwise provide a reproducible local/Docker setup.
- 2–3 minute demo video showing the repository, Swagger, successful request, failed request and architecture.

7. Time Limit

Maximum recommended time: 4–6 hours. Do not build a full CRM UI. The evaluation focuses on engineering decisions, backend fundamentals, AI integration, reliability and code quality.

8. Scoring — 100 Points

Category	Points
Python + FastAPI	15
Database design	10
AI / LLM integration	15
Structured AI output	10
CRM/business logic	10
Multi-tenancy / security	10
Background jobs / webhooks	10
Error handling / idempotency	10
Tests	5
Code quality / documentation	5
TOTAL	100

9. Shortlisting Guide

85–100: Strong candidate | **75–84:** Technical interview | **65–74:** Backup/consider | **<65:** Reject

Automatic rejection: no working API, no GitHub repository, copied/undemonstrated implementation, or inability to explain their own code.

10. Bonus Features

- Redis/ARQ or another proper queue

- Retry with exponential backoff
- LLM provider abstraction
- Prompt versioning
- Caching / duplicate-analysis prevention
- Docker / docker-compose
- Structured logging / observability
- Additional integration tests

Candidate instruction: Build only the feature described above. We care more about clean engineering, correct behavior, reliability and your ability to explain your implementation than visual polish.

Zizzet • AI Backend Developer Intern Screening